TrendPilot — a local agentic AI content assistant

AIRI Team PITB — AI Internship Task 2
Author: *Afrazia Umer* · Date: *13th September* · Submitted to: Omar Farooq, Associate AI/ML Engineer

> Every number in this report is read from `tests/results.json` and `outputs/memory.json`, which are produced by `python tests/run_tests.py`. Regenerate this document with `python report/build_report.py`.

---

1. Problem and project idea — *10 marks*

The problem. Turning a piece of work into content that people actually read is a separate skill from doing the work. An engineer who has just finished a project can describe it accurately and still write a post nobody stops scrolling for. The gap is not writing ability — it is knowing what angle to take, what hook opens it, what length the platform rewards, and whether the result is any good.

What TrendPilot does. It takes a topic, a platform and a tone, and returns a complete content plan: a chosen angle, a hook, a caption written to the platform's conventions, hashtags, optionally a video script and titles, a critique of its own output, and a saved file.

Why it runs locally. The whole thing runs on `gemma3:4b` through Ollama on the author's own machine. No API key, no per-token cost, no data leaving the laptop. That constraint shapes the engineering: a 4B model is far less reliable than a hosted frontier model, and most of the work in this project is the machinery that makes an unreliable model produce dependable output.

2. Ollama setup and local model — *15 marks*

Ollama version	0.34.1
Model used	`gemma3:4b` (3.3 GB)
Also tested	`llama3.1:8b` (4.9 GB)
Hardware	NVIDIA RTX 4050 Laptop, 6 GB VRAM
Placement	100% GPU, 4096-token context
Median time per full run	69.2s

```
ollama --version        # 0.34.1
ollama pull gemma3:4b   # 3.3 GB
python demo.py --check  # verifies Ollama, the model, and a test prompt
```

Why `gemma3:4b` over `llama3.1:8b`. Both were run on the same ten cases during development. gemma3:4b averaged roughly **108 s** per run against llama3.1's **145 s**, and the copy was not noticeably worse. On a 6 GB card the 4B model also sits entirely in VRAM, while the 8B model competes with anything else resident — see §7 for the failure that caused.

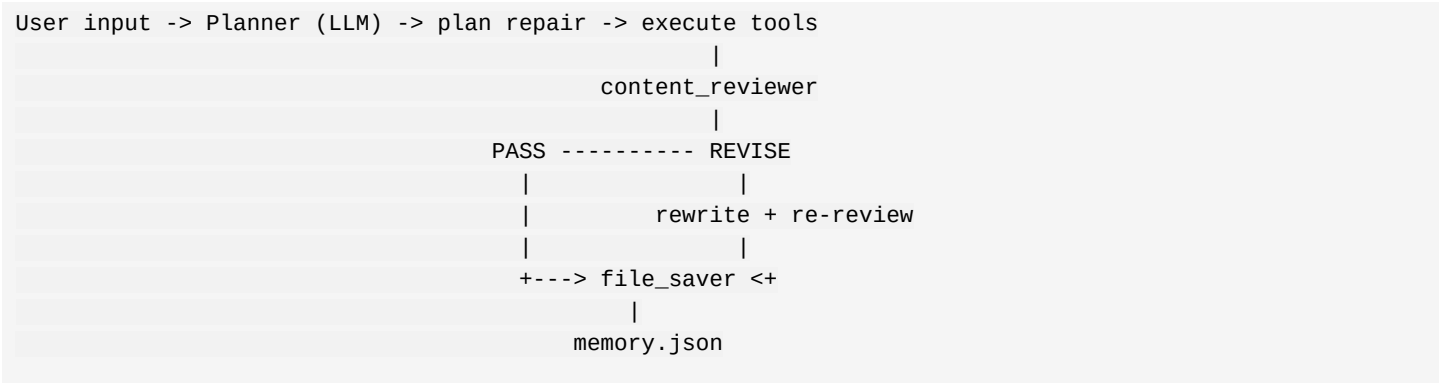
Screenshot required: `screenshots/ollama_running.png` — `ollama list` *and* `ollama run gemma3:4b` *with a test prompt.*

3. Agent workflow design — *15 marks*

The brief distinguishes an agent from a chatbot, and the distinction is easy to fail by accident: a script that calls five functions in a fixed order is a pipeline wearing an agent's clothes. Three behaviours here are genuinely agentic.

The agent chooses its own tools

The model is shown the tool menu and the user's request and returns a plan as JSON. Nothing about the order is hardcoded. Ask for *a post* and it omits the reel script generator; ask for *a reel* and it includes it.



Plans are repaired, not trusted

A 4B model given a tool menu will still invent `viral_maximizer`. `repair_plan` drops unknown tools, fuzzy-matches near misses, enforces the ordering the tools actually require, and guarantees `file_saver` runs last. If nothing usable survives it falls back to a default plan, so a bad plan never kills the run.

Across the 10 test cases the planner needed **1 repair(s)**.

It reviews and corrects itself

`content_reviewer` scores the output on six criteria and returns `PASS` or `REVISE`. On `REVISE` the agent rewrites the caption with the critique fed back, then re-reviews. It revises **at most once** — see §7 for why.

In testing, **9 of 10** runs triggered a revision.

4. Tool implementation — *20 marks*

The brief requires at least three tools. All eight are implemented.

Tool	Purpose	Calls the model?	Times used in testing
`memory_recall`	Recall previous topics and outputs	no	0
`trend_idea_generator`	Generate 5 angles, pick the strongest	yes	10
`caption_writer`	Write the platform-specific post	yes	9
`hashtag_generator`	Platform-appropriate hashtags	yes	8
`reel_script_generator`	4-scene, 45-second video script	yes	3
`title_thumbnail_generator`	Titles and thumbnail text	yes	3
`content_reviewer`	Critique, decide PASS or REVISE	yes	10

Tool	Purpose	Calls the model?	Times used in testing
`file_saver`	Write .md and .json (**mandatory**)	no	10

Two of the eight touch no model at all. `file_saver` writes to disk and `memory_recall` reads the store. That matters: an agent whose every tool is another LLM call has not really left the chatbot behind.

Each tool declares what it requires and what it produces, so a plan can be checked for executability before it runs rather than failing three tools in.

5. Memory and file saving — *10 marks*

Memory is a plain JSON store at `outputs/memory.json`, holding **56 runs** at the time of writing. A file the marker can open was judged more useful here than a SQLite database they cannot inspect without a client.

It is used for two things:

- `recent()` — what has been generated lately, shown by `demo.py --history`
- `similar_topics()` — Jaccard word overlap against past topics, so the agent can see it has covered something close before and avoid repeating the angle. Word overlap rather than embeddings: matching *"helmet detection project"* to *"YOLOv8 helmet detector"* does not need a vector database, and this adds no dependency and no model call.

Every run saves two files — a human-readable `.md` and the full run context as `.json`. **10 outputs were saved during testing** (the brief requires at least 5).

Screenshot required: `screenshots/saved_file_output.png` — a saved `.md` file open in an editor.

6. Demo interface — *10 marks*

Two demos are provided.

CLI (`python demo.py`) — interactive prompts for topic, platform, tone and output type, with `--check` to verify the environment and `--history` to show past runs. Every tool call is printed as it happens in the `[Tool Called]: name` format the brief specifies.

Streamlit (`streamlit run app/streamlit_app.py`) — the same agent with the trace streamed live into the page, so a viewer can watch it plan and decide rather than only seeing the output. Includes the reviewer's scores, the pre-revision caption, and a download button.

Screenshot required: `screenshots/demo_output.png`.

7. Testing and error analysis — *10 marks*

All ten cases from the brief were run on `gemma3:4b`. Full results: [`tests/test_cases.md`](tests/test_cases.md).

Cases run	10
Passed	**10/10**
Mean reviewer score	3.52/5
Runs that triggered a revision	9
Plan repairs needed	1

Tool errors	0
Median time per run	69.2s
Platforms covered	Instagram, LinkedIn, X, YouTube Shorts

Model call statistics. These are counted per process, so after a partial re-run (`--only`) they reflect that run rather than all ten cases:

Metric	Value
calls	83
repeat loop	1
seconds	471

Failures found, and what was done about them

Eight real failures, every one encountered while building and running this project rather than invented for the table.

#	Problem	Cause	Fix
1	The agent described the helmet project as a real-time rider alert system. It is not — it detects whether riders are wearing helmets.	The prompt forbade inventing *metrics* but said nothing about inventing *capabilities*, and a 4B model fills a four-word topic with plausible fiction.	Added a GROUNDING RULE to the system prompt that overrides everything else, and strengthened the reviewer's fabrication check to score 1 on any invented capability.
2	Revising made the output **worse** — the reviewer's own score fell from 4.0 to 3.3 after one revision.	The model shortened the post to address a length complaint and stripped the substance out with it.	Capped revision at one pass, and re-review after revising so the saved file reports the post that actually shipped.
3	`caption_writer` failed with HTTP 500 mid-run.	Ollama returns 500 while loading a model into VRAM. On a 6 GB card `llama3.1` (4.9 GB) and `gemma3:4b` (3.3 GB) cannot both be resident.	Retry server errors with backoff, fail fast on client errors. The run that followed had zero tool errors.
4	The planner returned tool names that do not exist.	A small model given a menu still invents plausible-sounding names.	`repair_plan` drops or fuzzy-matches them; `file_saver` is appended unconditionally.
5	Fuzzy matching sent `caption_generator` to `hashtag_generator`.	Every tool name ends in `_generator` or `_writer`, so the shared suffix outweighed the meaningful word in `difflib`'s character-level ratio.	Weighted the head word decisively. Verified on 7 name-matching cases.
6	A request for a **short post** on X produced a 4-scene video script — twice.	The check for whether the user wanted video matched the substring `short`, intended for *YouTube Shorts*, inside *short post*.	Whole-word matching, with `shorts` (plural) as the video signal. Verified on 9 cases; both affected tests then passed.

#	Problem	Cause	Fix
7	Re-running two cases with `--only` **deleted the other eight results**.	The test runner wrote a fresh results file rather than merging into the existing one.	Partial re-runs now merge, replacing only the cases just run.
8	Both video cases failed with HTTP 500 even after three retries. The real error, hidden in the 500's body: `prediction aborted, token repeat limit reached`.	gemma3:4b falls into a repetition loop on nested-JSON prompts. Retrying an identical request cannot help. Measured: `temperature 0.7` loops, `0.95` does not; `num_predict` is irrelevant and `repeat_penalty` alone does **not** fix it — which is counterintuitive, since that is the parameter whose name suggests it should.	Read the error body, detect the repeat message, and retry with temperature raised by 0.2. Both cases then passed, and the fix is recorded firing twice in `llm_stats.repeat_loop`.

Three improvements, in priority order

1. **Make the reviewer independent of the writer.** The same model both writes and grades, so it is marking its own homework — which is exactly why it is weakest at catching fabrication. Running the review on a different local model would cost one extra load and remove the conflict.
2. **Let the user supply project details, not just a topic.** Most fabrication traces back to a four-word topic the model had to pad. An optional details field would remove the need to guess.
3. **Only accept a revision if the re-review scores higher.** The machinery already re-reviews; it just does not yet compare. Keeping the better of the two would have prevented failure 2 entirely.

8. What I learned

- **Most of the work was not the agent, it was the scaffolding.** The plan/route/review loop is maybe eighty lines. The JSON repair, plan validation, retry logic and error containment are several times that, and without them a 4B model produces an unusable system.
- **"Agentic" is easy to fake and easy to fail by accident.** Writing a fixed sequence of tool calls would have looked identical from the outside and been a pipeline. Letting the model choose, then repairing what it chose, is the part that makes it an agent.
- **A small model's failure modes are specific and learnable.** Markdown fences around JSON, trailing commas, truncation at the token limit, invented tool names. Each has a deterministic fix. None needed a bigger model.
- **Self-review is not self-improvement.** The reviewer reliably catches length and tone, and reliably misses fabrication — the failure that matters most. And a revision it demanded made the post measurably worse.
- **Local inference is slow enough to change design decisions.** At one to two minutes per run, every extra tool call is felt, which is why the agent skips tools the user did not ask for rather than always running all eight.

9. Deliverables

- [x] Complete source code (`app/`, `demo.py`)
- [x] `requirements.txt`
- [x] README
- [x] Working demo — CLI and Streamlit
- [x] Saved generated outputs (10, brief requires 5)
- [x] Test case results (10 inputs)
- [x] Error-analysis table
- [x] Final report PDF

- [x] LinkedIn / portfolio section
- [x] Screenshot: Ollama running
- [x] Screenshot: demo output
- [x] Screenshot: saved file
- [ ] GitHub / Drive link

Generated 2026-09-28 07:14 by `report/build_report.py`.